School of Computing Science

CSC2101 – Professional Software Development & Team Project 1

Customer Day 2 Progress Report

Assoc. Prof. Cao Qi
Qi.Cao@Glasgow.ac.uk

Project Overview

The project aims to increase event feedback response rates by 40 - 60% through the use of gamified elements, while also improving the quality and depth of feedback collected from participants. By incorporating engaging interactions, the solution seeks to enhance participant involvement and create a positive post-event experience that encourages continued engagement. In doing so, it will provide the Singapore Cancer Society (SCS) with actionable insights to refine and improve future events, while simultaneously strengthening connections with the community and supporting the organization's mission. Additionally, the incentive-based design will motivate users to return and participate in subsequent events, fostering long-term engagement.

Scope

The project involves designing and developing a gamified feedback system with the following key features:

- Design and develop interactive feedback form with gamification features.
- Mobile-responsive web application accessible via QR code or links.
- Basic Analytics Dashboard for Staff from Singapore Cancer Society.
- User testing and iteration based on feedback.
- Training materials for event organisation.
- Linking external databases with the game so that users can change variables in the game using the database.

Completed Deliverables

Our team has successfully completed several key deliverables as part of the project's initial development phase. We have produced a first working concept and UI flow of the gamified feedback application, initially centered around a virtual pet system to encourage participant engagement through checklists and incentives. Following client feedback from both the Zoom consultation and the recent in person meeting on CD02, we revised and presented an updated UI design and flow, refined design direction, simplifying the experience into a single conversational style feedback survey with integrated event checklists and lighthearted visual elements similar to the Pokemon games.

In addition, we developed a detailed software architecture diagram outlining the integration between the frontend and backend systems (using Render for hosting, Supabase for database, etc.). The team also produced six well-defined user stories, conducted Planning Poker estimation sessions to assign story points and compiled a comprehensive product backlog ranked by priority. From this backlog, two top priority user stories, Gamified Survey and Insight Dashboard were selected. These stories were further broken down into specific sprint tasks with estimated hours and assigned team roles.

The team has also added some of the client requests into the backlog, such as adding raw data export and event impact analytics to the admin dashboard and aligning the design tone to be more friendly and approachable.

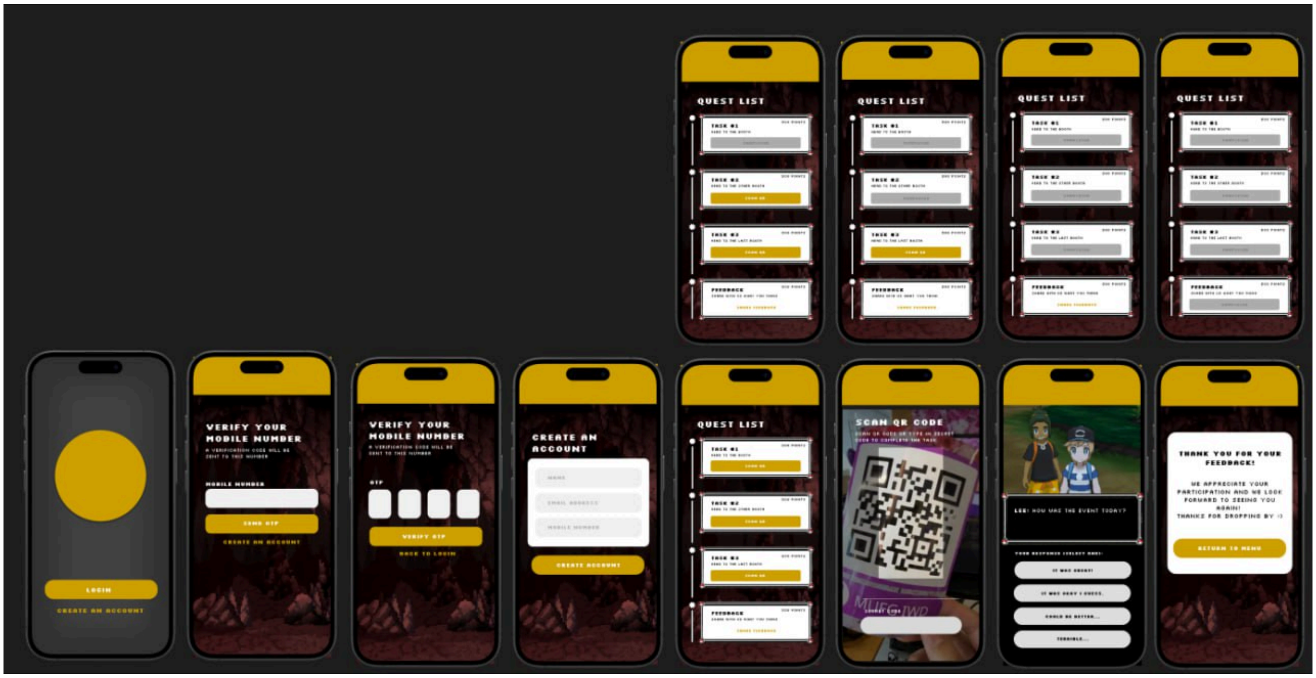
Activity Diagram

See figure 1.1

Deliverables

1	Fully Functioning Gamified Feedback Platform
2	Mini-Storyline or Avatar-Based Flow
3	Multilingual Support
4	Administrative Dashboard (Viewing/Managing Questions, Responses and Analytics)
5	QR Generated Tools for events
6	User Testing, Performance and Engagement Metric report
7	Implementation Guide & Technical Documentation

Prototype Design



Core Features:

- Quest/Task System - Seniors complete social activities and tasks for points
- Authentication - Login, OTP verification, account creation
- Social Connection - QR code scanning to connect with others
- Gamification - Points, avatars, rewards to encourage participation

Design Focus:

- High contrast (dark background + yellow accents)
- Large buttons and text for accessibility
- Card-based interface for easy navigation
- Designed specifically for senior users to make social engagement fun and simple

Timeline Breakdown

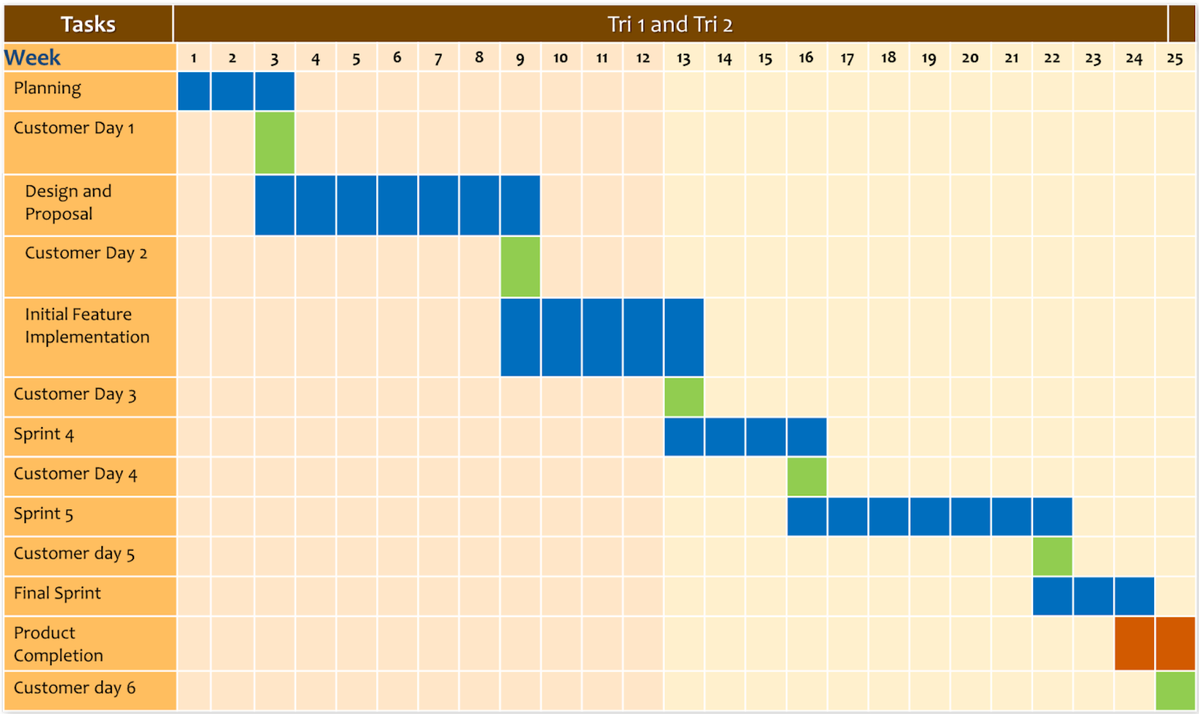


Figure 1: *Proposed Timeline*

Phase	Duration	Description
Planning Phase	Week 1 - 2	Initial project setup and requirement gathering.
Design and Proposal	Week 3 - 9	Extend design phase with customer validation checkpoint.
Implementation Cycles	Week 10 - 22	Multiple sprint cycles including initial feature implementation, Sprint 4, Sprint 5, each followed by customer review sessions.
Final Phase	Week 23 - 25	Final Sprint, Product Completion, and concluding Customer Validation.

Figure 2: *Phase Breakdown*

Project Backlog

Following feedback gathered during Customer Meeting Day 2, the team has selected two high-priority user stories from the product backlog for Sprint 3 development. These features focus on enhancing the attendee experience through a gamified survey and delivering actionable insights for organizers via a dashboard.

First User Story: Gamified Survey

Description:
As an attendee, I want to complete a short, gamified feedback survey that awards points and a badge upon completion, so that sharing my experience feels fun and rewarding.
Acceptance Criteria:

- Survey displays a progress bar and estimated completion time.
- Participants receive points and a badge upon successful submission.

Task	Estimated Time (hrs)	Assigned Member
Frontend Implementation - Build js components for gamified survey	10	Wymen Lim
Backend API Implementation - Implement REST endpoints for survey and rewards	12	Dereck Zorca
Integration and API Linking - Connect frontend with backend REST APIs	8	Azry Delvin
Testing of the survey	6	Subhashini Rajkumar
Documentation and Demo	4	Azry Delvin

Total Estimated Time: 40 hours

Second User Story: Insight Dashboard

Description:
As an SCS organizer, I want a dashboard summarizing response rates, completion time, sentiment, themes, and NPS so that I can extract actionable insights to improve future events.

Acceptance Criteria:

- Dashboard cards display response rate, depth boost uptake, and average completion time.
- Data updates dynamically from live survey submissions.

Task	Estimated Time (hrs)	Assigned Member
Dashboard Design - Create wireframes for dashboard layout and cards	6	Wymen Lim
Frontend Dashboard Development - Build dashboard	10	Wymen Lim
Analytics API Development - Develop Python APIs for analytics	12	Dereck Zorca
Database Modelling - Define SQL/NoSQL schemas and optimize queries	6	Azry Delvin
Testing- Validation of accuracy of metrics and visuals	6	Subhashini Rajkumar
User Guide – Document usage and interpretation of dashboard metrics	3	Subhashini Rajkumar

Total Estimated Time: 43 hours

Resource Allocation

Tools	Purpose	License	Reason
Next.js	Full-stack framework	Open Source (MIT)	App Router, server actions, RSC
React	UI library	Open Source(MIT)	Latest concurrent features
Tailwind CSS	Styling	Open Source(MIT)	Rapid UI development
NextAuth.js	Authentication	Open Source(MIT)	Build for next js
Animation Creation	Figma	Proprietary (Free tier)	Industry standard and Library of pre-made components

Technology Stack Overview

The gamified event feedback platform is built on a modern, full-stack technology foundation consisting of five core tools, all utilizing free or open-source licenses to maintain zero infrastructure costs.

Next.js serves as the primary full-stack framework (MIT License), selected for its integrated App Router, Server Actions, and React Server Components that eliminate the need for separate frontend and backend codebases. This unified architecture significantly reduces development complexity and accelerates implementation.

React (MIT License) provides the UI library foundation, chosen for its latest concurrent rendering features that enable smooth, responsive user interactions essential for the gamified experience. As the industry-standard component library, React ensures access to extensive documentation, community support, and third-party integrations.

Tailwind CSS (MIT License) handles all styling needs through its utility-first approach, enabling rapid UI development without context-switching between component files and stylesheets. This approach is particularly valuable for creating custom gamification interfaces like character cards, progress bars, and animated feedback elements.

NextAuth.js (MIT License) manages authentication and user sessions, specifically built for Next.js applications. It provides secure, zero-configuration integration with OAuth providers and traditional email/password authentication, eliminating the need to build authentication infrastructure from scratch.

Risks

Technical Risks
Gamification fatigue - Users may find the quest system will be tiring to follow through as they would have to finish all the stages to unlock the feedback form.

Mobile Compatibility Issues - Event attendees will likely use various devices with different screen sizes, browsers and capabilities. QR code scanning and real-time check-ins need to work reliably across all platforms

Network Connectivity - Events may have poor WIFI or cellular coverage, causing failures in both check-ins or feedback submission. Need offline capability or graceful degradation.

Scalability - Large events with hundreds of simultaneous users checking in at booths could overwhelm the system if not properly architected.

User Experience Risks
Complexity Overload - Combining event tracking + booth check-ins + gamified feedback might overwhelm users, especially elderly attendees or those less tech-savvy

Privacy concerns - tracking attendee movements and collecting feedback data requires careful handling of personal information and clear consent mechanisms.

Adoption Resistance - Attendees accustomed to traditional feedback methods

Organizational Risks
Maintenance Burden - A custom gamified platform requires ongoing maintenance, updates and support that may not have resources for long-term.

Tutorial Questions

There are different templates can be used for data gathering in the sprint retrospective meeting. In the second exercise, we will choose the template of Start, Stop, Continue, to learn how to conduct the sprint retrospectives.

It typically takes around 10-25 minutes to conduct a “meta” process discussion. Each team member will populate a template board independently using sticky notes (or write on a paper by each member). The following questions will be used for your data gathering:

- What should we start doing that we aren't doing yet?
- What did we do that did not help and we should stop?
- What did we do well and should keep doing, or do more of?

Complete the following board with at least 10 items for each category.

Start	Stop	Continue
- Start coding the front end logic - code out the backend logic - Test out the database - Daily standup meetings - Distributing the workload - Write unit testing - Code reviews before merging - Documentation as we build - Using the kanban board - Pair programming for complex features	- Stop working on the prototype - Stop having unstructured meeting - Stop last-minute code commits - Stop working in silos - Stop ignoring technical debt - Stop multi-tasking across multiple features - Stop skipping code reviews - Stop unclear task assignments - Stop unrealistic sprint commitments - Stop informal requirement changes	- Weekly standup meeting - Check-in with the client - Taking notes of the requirements and feedback from the client - Using git - Sprint planning sessions - Collaborative problem solving - Regular code commits - Testing before deployment - Using project manager tools - Celebrating small wins

We have learnt the Five Whys, a technique for diagnosing the root cause of an issue in the project. In the third exercise, we will employ the Five Whys technique to explore the root causes of a problem or defect in the last sprint of your CSC2101 – TP projects. The key is to explore cause-and-effect relations by repeating the question "Why?" five times. In general, five iterations will likely help to get a root cause.

Identify top FOUR (4) problems in your last sprint and follow the below steps.

Problem #1:	No working increment at the end of the sprint
Why?	We do not have an idea of what kind of product to make yet

Why?	There were still uncertainties about the requirements from the customer
Why?	We were waiting for a second meeting with the customer to clarify our doubts
Why?	The uncertainties only arose after brainstorming for ideas after the first customer meeting
Why?	We failed to anticipate and cover potential difficulties or uncertainties during the first customer meeting.

Problem #2:	Unsure of how to split workload among our team
Why?	We are unfamiliar with how to split the workload efficiently to avoid conflicting or overlapping with each other
Why?	We lack the experience and foresight in anticipating how different tasks may depend on each other
Why?	This is our first time working together as a team to develop a software
Why?	We are still new to practical project management and task coordination
Why?	

Problem #3:	Difficulty estimating task duration and effort
Why?	Many tasks took longer than originally planned
Why?	We are not able to accurately estimate how much time is needed to work with new and unfamiliar technology
Why?	We do not have any experience in developing a software that will be deployed for real-world use
Why?	We do not have past experience to base our estimation off of
Why?	This was our first time working together so we are not familiar with how fast each other work

Problem #4:	Our scrums meetings was not very effective

Why?	Our scrum meetings take a long time to achieve our goal for that meeting
Why?	We are unsure of how and what details to share about our progress
Why?	There was no meeting structure for our scrum meetings
Why?	The meeting was not facilitated well by our scrum master
Why?	This was our first time following the scrum framework and we were unfamiliar with it

Appendix

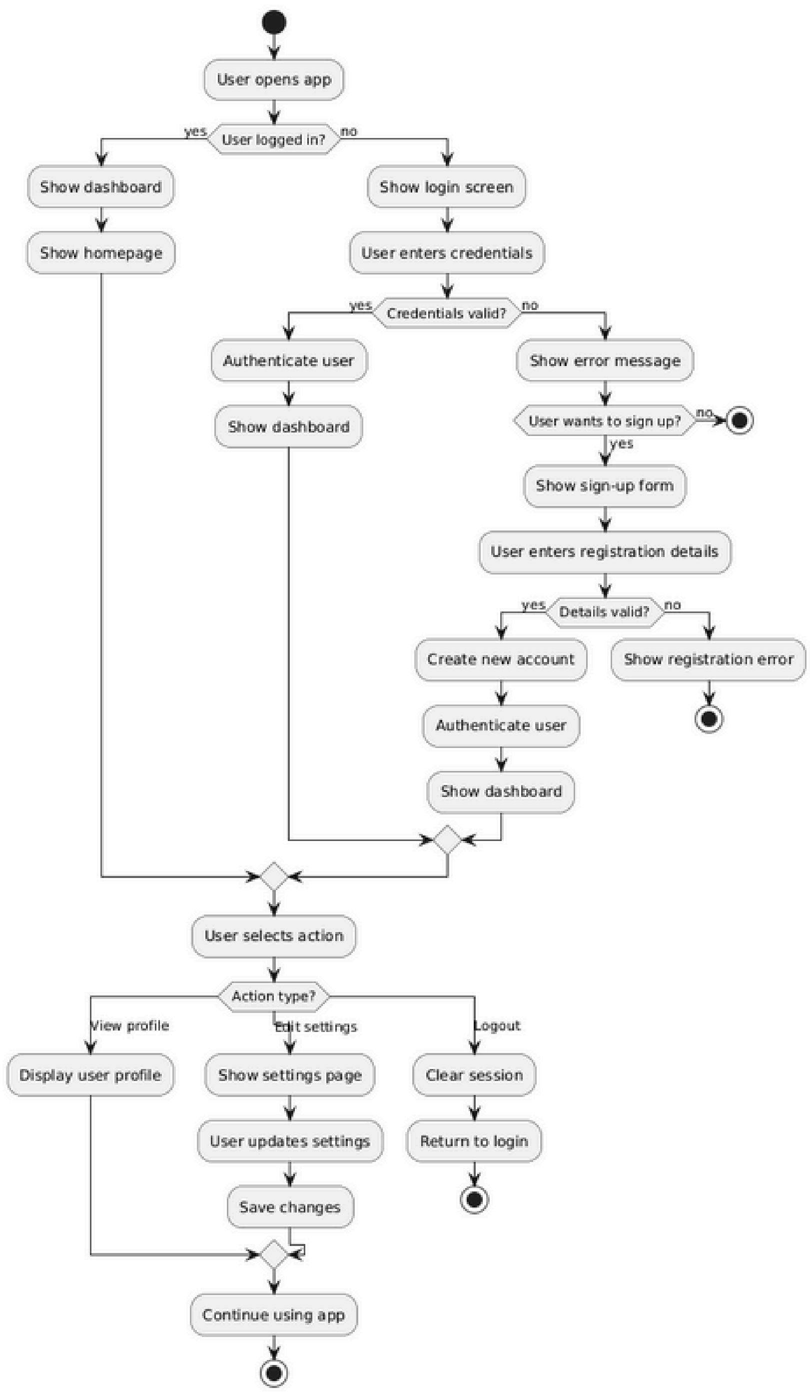


Figure 1.1